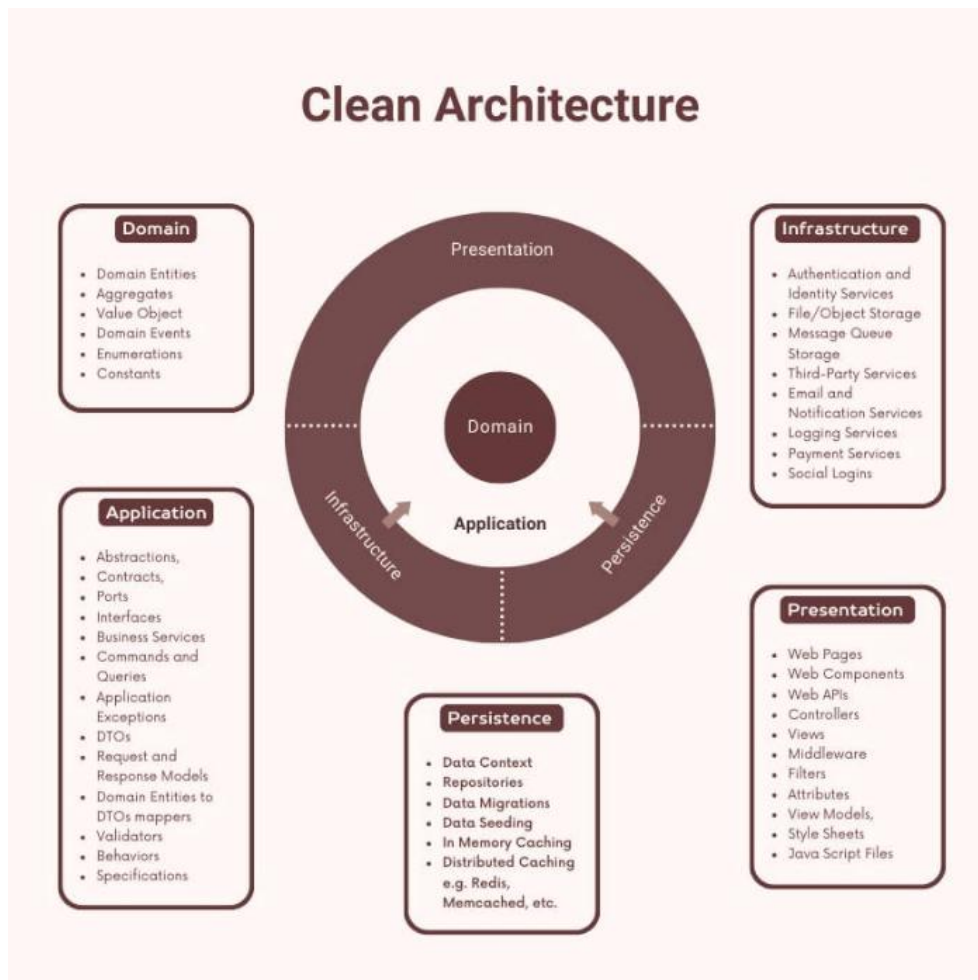


Föreläsning – 260622 – ASP.NET CORE WEB.API

System-arkitektur – Fortsättning

Olika Systemarkitekturer:

Clean Architecture

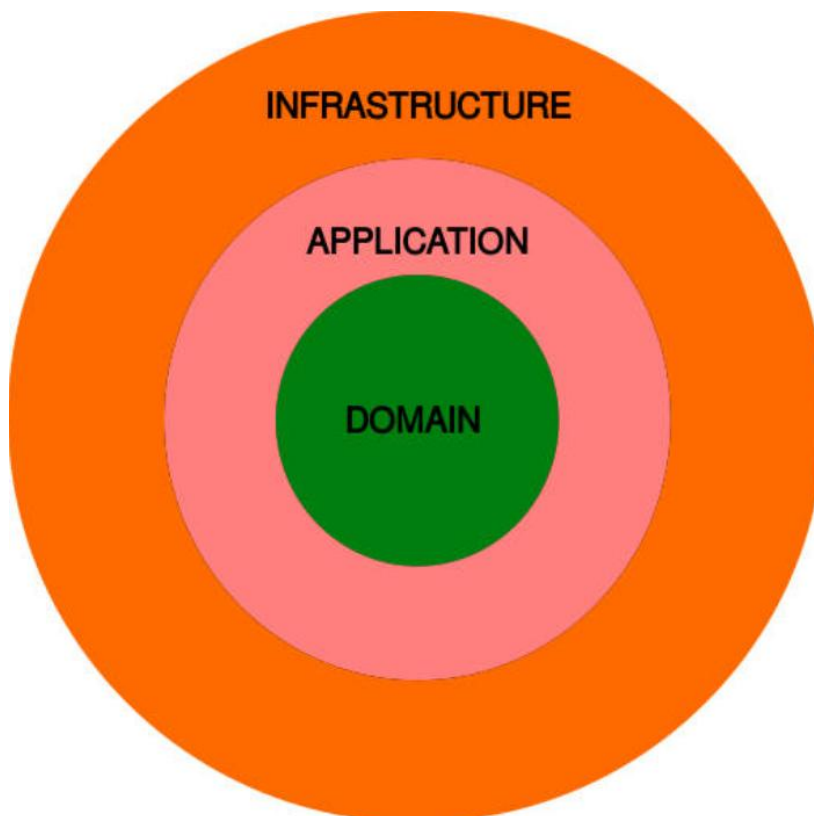


Clean Architecture är domäncentrerad och brukar organiseras med ett kärnprojekt för domain och use cases, medan ytterlager hanterar webb, databaser och andra tekniska detaljer. **Grundtanken är att kärnan ska vara oberoende av ramverk och infrastruktur**, vilket gör systemet mer testbart och lättare att ändra över tid.

Passar bra när:

- Domänlogiken är viktig och vill skyddas.
- Man vill kunna **byta databasteknik eller UI** utan att röra kärnan.
- Man prioriterar testbarhet och långsiktig underhållbarhet.

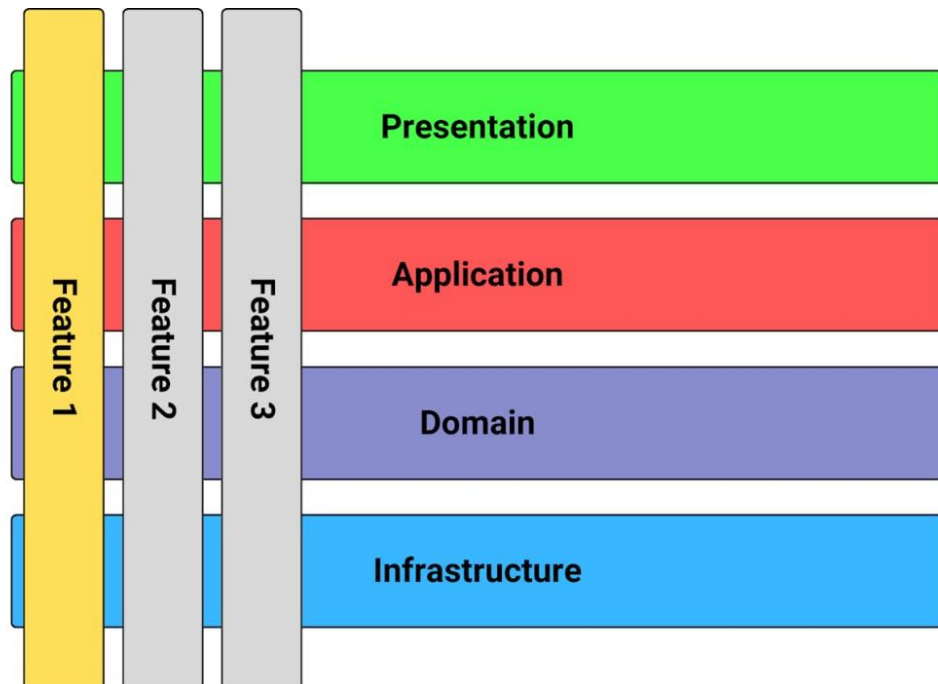
Onion architecture



Liknar **Clean Architecture** mycket, med domänen i mitten och tekniska beroenden längre ut i cirkelarna. Den betonar samma princip: innersta lagret ska inte känna till ytterlagren, och allt riktas mot kärnan.

Används ofta när man vill ha en tydlig mental modell för beroendestrukturen, särskilt i lösningar med **mycket affärslogik**.

Vertical Slice architecture



Vertical Slice Architecture organiserar systemet per feature eller use case i stället för per tekniskt lager. Varje slice innehåller allt som behövs för just den funktionen, till exempel endpoint, request/response, handler, validering och datatillgång.

Exempel på struktur:

- Features/Orders/CreateOrder
- Features/Orders/GetOrderById
- Features/Customers/SearchCustomers

Layered architecture

Delar upp lösningen i API/UI, service/application, domain och infrastructure. Den är enkel att förstå och fungerar bra för mindre till medelstora API: er, men kan bli komplicerad när applikationen växer och många features börjar korsa samma lager.

Typiskt upplägg:

- API/Controllers.
- Application/Services.
- Domain/Models.
- Infrastructure/Repositories, EF Core, externa integrationer.

Den här modellen är ofta bra som startpunkt när teamet vill ha tydlig struktur utan att överkomplicera lösningen.

Modular monolith

Är fortfarande en monolit i deployment, men uppdelad i tydliga moduler med egna gränser, ofta runt affärsområden som **Users** och **Payments**. Varje modul kan innehålla sin egen domain, application och infrastructure, men systemet körs som en enda applikation.

Bra när:

- Man vill undvika mikroservices-komplexitet.
- Man har flera affärsdomäner som bör vara tydligt separerade.
- Man vill kunna växa mot tjänsteorientering senare.

Den här modellen är ofta ett starkt val för **större Web API-projekt** som ännu inte behöver distribueras som mikroservices.

Hexagonal architecture

Hexagonal Architecture, eller ports och adapters, fokuserar på att kärnan exponerar portar som yttervärlden ansluter till via adaptrar.

I praktiken ligger tanken nära Clean Architecture: domänen skyddas, och infrastruktur som EF Core, REST, messaging eller externa API: er blir utbytbara adaptrar.

I ASP.NET Core betyder det ofta att controllers eller endpoints är adaptrar in mot use cases, och att e-post, köer och databaser är adaptrar ut från kärnan.

Val av arkitektur

Arkitektur	Styrka	Nackdel	Passar bäst för
Layered	Enkel och välkänd	Kan bli tung och korskopplad	Små till medelstora API:er
Clean Architecture	Testbar och domänfokuserad	Mer struktur och fler projekt	Affärskritiska system
Onion	Tydlig beroendecirkelform	Nästan samma som Clean i praktiken	Domänorienterade system
Vertical Slice	Feature-fokuserad och praktisk	Kräver disciplin kring gränser	API:er med många use cases
Modular Monolith	Bra modulgränser utan distributed complexity	Kräver bra moduldesign	Större system som ska hållas enkla
Hexagonal	Flexibelt och teknologineutralt	Kan upplevas abstrakt	System med många integrationer